

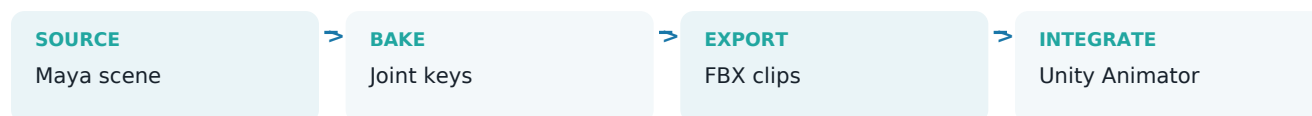
Maya to Unity

Animation Pipeline

A production-oriented guide for exporting a reusable character rig and animation clips from Autodesk Maya into Unity.

Purpose

This workflow separates the character asset from its animation clips so one rig can support many animations without duplicating the mesh and materials in every file. It is written as a concise technical-art reference: clear enough to follow, but structured around the constraints that keep a Unity project maintainable.



1. Prepare Clean the export skeleton and preserve a consistent hierarchy.	2. Bake Convert controller-driven motion into joint keyframes Unity can read.
3. Export Create one base-model FBX and one FBX per animation clip.	4. Integrate Configure import settings and build the Animator state machine.

Core principle

The base-model FBX defines the mesh and skeleton. Each animation FBX contains only baked joint animation. Keep the same root and joint hierarchy across every export.

Choose the right workflow

A single-file export can be useful for a quick prototype. For a character with multiple clips, use the separate model-and-animation workflow below.

	Single-file export	Reusable clip pipeline
Best for	One-off tests or a single animation.	Production characters with multiple states.
Files	Each file includes asset data and animation.	One model FBX plus a lightweight FBX per clip.
Tradeoff	Fast to set up, but duplicates data.	More setup, but easier to scale and update.

Before you export

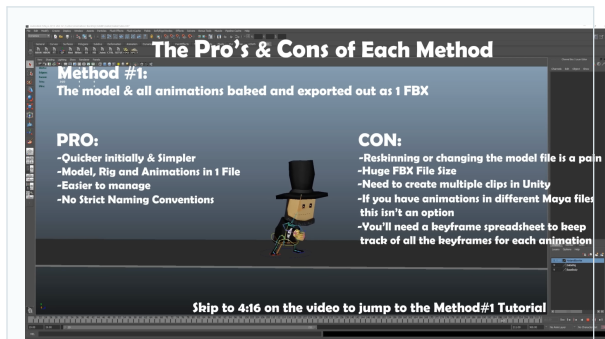
- Confirm the character plays correctly in Maya and the intended clip range is known.
- Use a stable root joint at the top of the skeleton. Do not rename, add, remove, or reorder joints between clip exports.
- Remove scene-only content from the export: lights, cameras, geometry not needed by the asset, NURBS curves, IK handles, and animation controllers.
- Keep a saved Maya working scene with the full rig. Animation export scenes are disposable outputs, not your source of truth.

Important

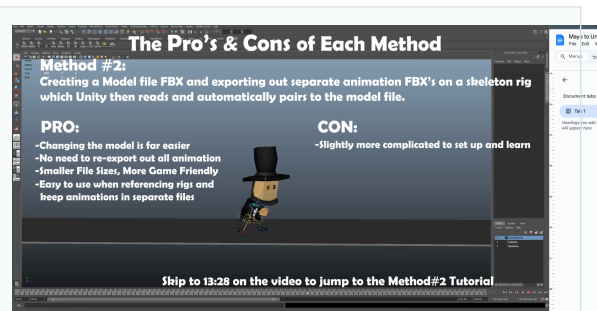
Unity imports joint animation, not Maya rig logic. Bake the animation before exporting so controller- and IK-driven motion is stored as keys on the skeleton.

Visual workflow reference

These production screenshots document the source workflow: the export-method tradeoffs, the Maya baking process, and the Unity-side integration point.

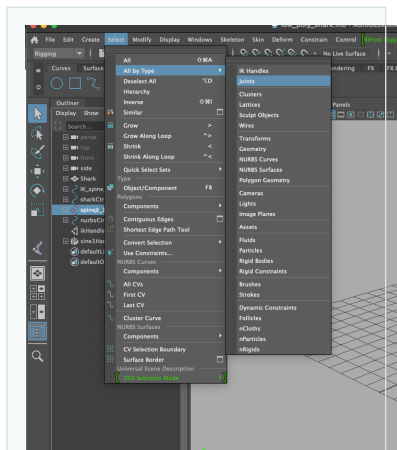


Reference: Method 1 tradeoffs - a compact, single-file approach.

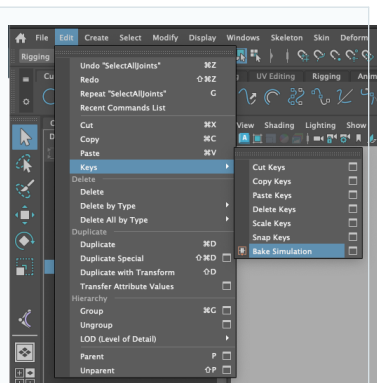


Reference: Method 2 tradeoffs - reusable clips with a shared character asset.

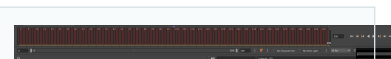
Maya export and bake evidence



Select the approved export set.



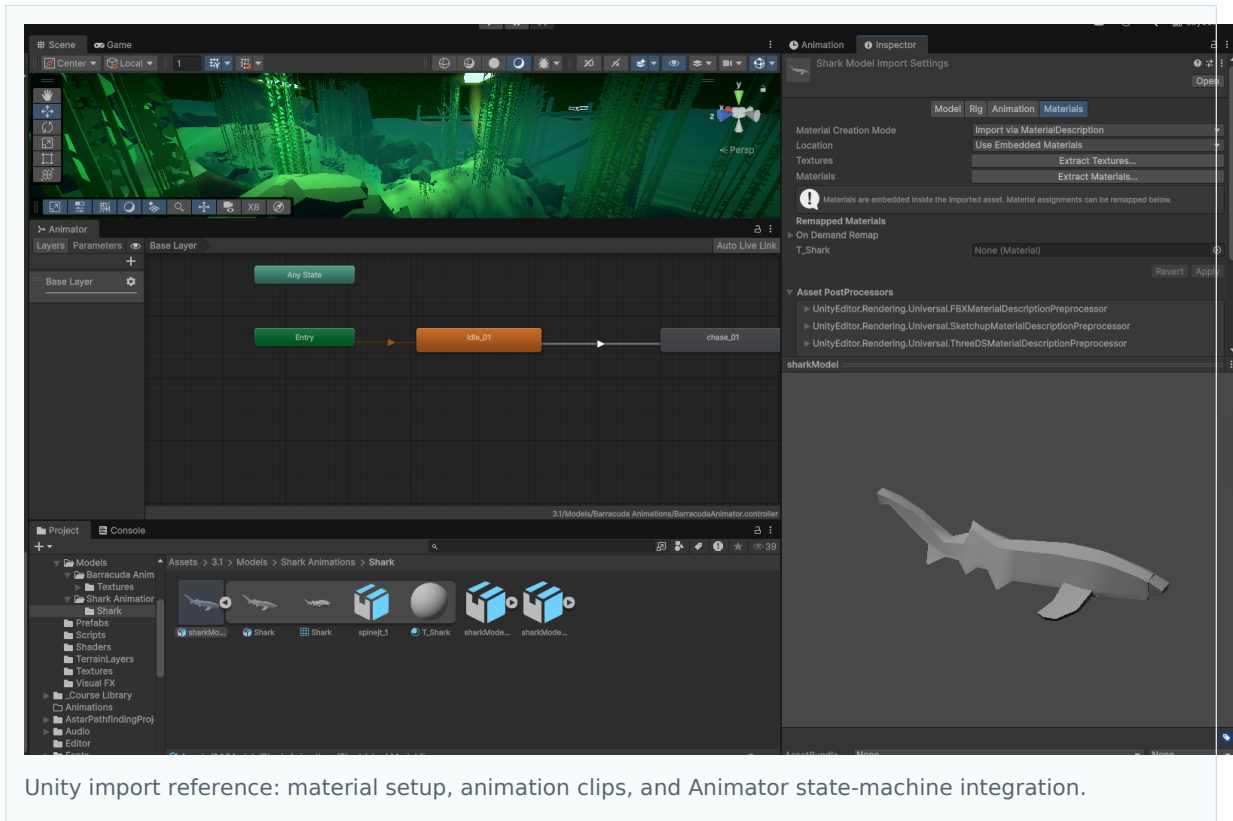
Bake simulation onto joints before FBX export.



Confirm the timeline contains baked keys.

Unity integration evidence

The final handoff happens in Unity: confirm the character import, assign materials, and connect the imported clips in an Animator Controller.



1. Export the base model

The model FBX is the reusable character asset. It contains the mesh, skeleton, and material data, but no animation clips.

In the Maya source scene

- 1 Choose File > Export All (or export only the approved asset set).
- 2 Export as <ModelName>.fbx with animation disabled.
- 3 Open the resulting FBX in a fresh Maya scene.
- 4 Keep only the renderable mesh and skeleton. Remove NURBS, IK controls, control curves, extra cameras, lights, and environment content.
- 5 Verify the root joint is at the top of the hierarchy. This hierarchy must remain identical for every animation export.
- 6 Re-export and replace <ModelName>.fbx.

Suggested project layout

Assets/Characters/Barracuda/
Models/Barracuda.fbx
Animations/Barracuda@Idle.fbx, Barracuda@Chase.fbx, Barracuda@Sprint.fbx
Materials/ and Textures/

2. Export animation clips

Each clip is exported from the same source rig after the animation has been baked onto its joints.

Do not overwrite your source scene

Start in the main Maya scene, export the clip, and use a new scene to inspect or trim the exported FBX. Do not save changes over the original rig scene.

Bake and export

- 1 In the original Maya scene, select all joints: Select > All by Type > Joints.
- 2 Bake the motion: Edit > Keys > Bake Simulation. The timeline should now show keys on the skeleton.
- 3 Select the joints again and choose File > Export Selection.
- 4 Enable animation in the FBX export options.
- 5 Name the file <ModelName>@<AnimationName>.fbx. Example: Barracuda@Idle.fbx.
- 6 Repeat for each animation state.

Trim a clip when needed

- 1 Open the exported animation FBX in a new Maya scene.
- 2 Set the timeline to the exact start and end frames for that clip, and remove unrelated animation data.
- 3 Select the entire skeleton and export selection again with animation enabled, replacing that clip FBX.

Why the @ naming pattern helps

Unity uses the model name before @ to associate an animation file with its matching model. Consistent names reduce manual remapping and make the asset relationship readable in the Project window.

3. Integrate and validate in Unity

Once the FBX files are organized, Unity can import the character and its clips as a coherent asset set.

Import and configure

- 1 Place the model FBX and animation FBXs in the same character folder under Assets.
- 2 Select the model in Unity and confirm its Rig settings match the character type. Apply the import settings.
- 3 For each animation FBX, inspect the Animation tab. Confirm the intended clip range and enable looping only for looping states such as Idle, Swim, or Run.
- 4 Import textures and create Unity materials. If the Maya scene uses Arnold materials, rebuild the material in the project render pipeline and connect the appropriate texture maps.
- 5 Create an Animator Controller: right-click in the Project window, then Create > Animator Controller.
- 6 Open Window > Animation > Animator and drag each clip into the graph. Add transitions and parameters that match gameplay behavior.
- 7 Add an Animator component to the scene instance and assign the controller.

Final validation checklist

- Mesh, scale, orientation, and materials look correct in the Unity scene.
- Every clip is bound to the intended avatar/skeleton and plays through its full range.
- Looping clips return cleanly to their first pose; one-shot clips do not loop.
- Animator transitions are reachable and return to expected states.
- No missing textures, duplicate clips, or unintended scene objects were imported.

Portfolio takeaway

This pipeline demonstrates technical-art thinking beyond a simple export: rig compatibility, baked-data constraints, asset naming, reusable clip organization, and runtime animation-state integration.